Week 9, Lecture 2: Hamiltonian Monte Carlo and Stan

Instructor notes. Part 1: prep reading. Part 2: podium cheat sheet

How to prepare (for students)

- **Big picture.** On Monday we turned simulated annealing into the Metropolis algorithm, and in Lab 7 you saw its weak spot: on a long, narrow, tilted posterior a random walk creeps, and you had to reparameterize by hand to fix it. Today: Hamiltonian Monte Carlo (HMC), the method behind **Stan**, which uses the *slope* of the posterior (the same gradient Newton's method used in Week 8) to make long, well-aimed moves. Stan is one of the most widely used tools for fitting custom Bayesian models in ecology, and we'll use it for the rest of the semester.
- **Read:** Monnahan, Thorson & Branch (2017), "Faster estimation of Bayesian models in ecology using Hamiltonian Monte Carlo," *Methods in Ecology and Evolution* 8: 339-348 (focus on the introduction and the comparison with JAGS; skim the details). Optional: Bolker 7.3.3 (Gibbs sampling and BUGS, printed p. 315-319), and McElreath, *Statistical Rethinking* (2nd ed.), Ch. 9, for the friendliest picture of HMC.
- **Review:** Monday's Metropolis rule and the myxomatosis Gamma model; what a gradient is (Week 8).
- **Install Stan before Wednesday if you can** (the take-home WOD walks you through it): https://mc-stan.org/cmdstanr/articles/cmdstanr.html.

Overview (instructor only)

Beat	Mode	Time
0. Lab 7 debrief: which chains to trust?	R	~3 min
1. Big picture	BOARD	~3 min
2. Why random walks struggle	BOARD → R	~7 min
3. Hamiltonian Monte Carlo	BOARD → R	~10 min
4. Stan: the myxo model	R	~12 min
5. When it fails; Stan vs. JAGS	BOARD	~4 min
6. Wrap, where to learn more	BOARD	~2 min
Discussion	BOARD	~5 min

No in-class WOD. Hand out the take-home WOD (*Getting Stan Running*, due Monday) at the end. **BOARD** = whiteboard; **R** = the Wednesday section of

`CODE_Week9_MCMCAndStan.R`. It reuses Monday's functions, so run the Monday section first (a few seconds). **Compile the Stan model before class**: it took 8 s when these notes were knit, but a first compile can take up to a minute. Sampling takes about 2 s.

Why myxo, not Lab 7's chytrid model. Lab 7 Act 2 may still be unfinished at home, and a Stan fit of that model would give away its credible interval. Myxo is Monday's banana, so the comparison with Metropolis is direct.

Where section 0 came from. Monday's closing discussion (three germination chains) wasn't reached. Lab 7 Act 1 Part 2 then asked the same question with four chains, so it opens today as a short debrief.

If running behind, skip the hand-coded HMC run in section 3 (draw the picture instead) and shorten section 5 to the two bullets on divergences and discrete parameters.

Part 1: Lecture content

0. Lab 7 debrief: which chains to trust? (~3 min) [R]

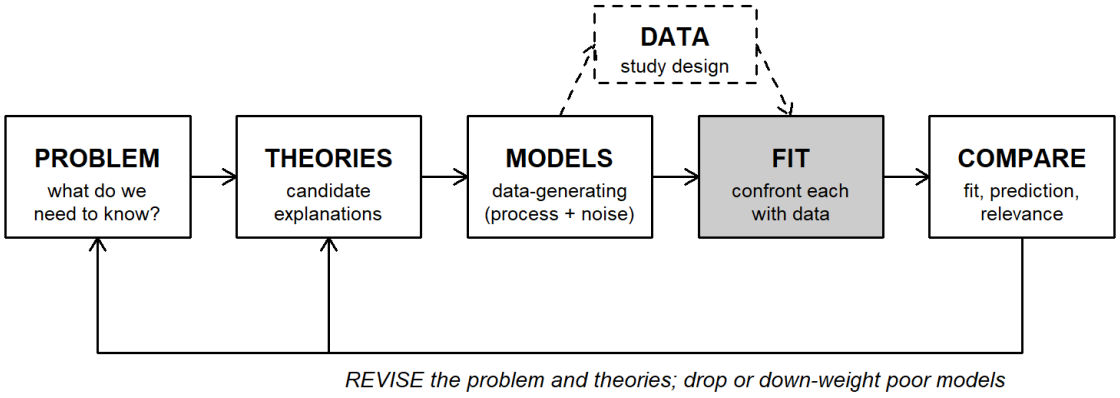
Run section 0 of the script. Same germination data as Monday (30 of 40, flat prior, exact posterior mean 0.738), same start ($p = 0.05$), 1,000 steps:

proposal sd	accepted	chain mean
0.005	87%	0.38
0.1	59%	0.73
3	4%	0.70

Ask: "A labmate says the first is best, since nearly every proposal was accepted. Which Lab 7 chain does each one look like?" Then show the trace plots.

Where to land (name the terms groups were reaching for in Lab 7): tiny steps are nearly always accepted but barely move, so the chain is still climbing (it never finished **burn-in**; like Chains B and D). Huge steps are nearly always rejected, so the chain sits still (like Chain C); its mean is close by luck. The middle chain **mixes** well. Acceptance rate is not a quality score: check the trace plot, drop the burn-in, run more than one chain, and tune the step size. Segue: even a well-tuned random walk creeps when the posterior is a narrow, tilted ridge (Act 2's raw temperature), which is today's topic.

1. Big picture (~3 min) [BOARD]



- Point at **FIT**. Monday: Metropolis turns a posterior into samples without computing $P(y)$. Lab 7: it works, but on correlated or many-parameter posteriors it creeps, and you have to help it.
- Today: a sampler that uses the shape of the surface, and a tool (Stan) that builds it for you from a model description.

2. Why random walks struggle (~7 min) [BOARD → R]

Board. Sketch a long, narrow, tilted ellipse (Lab 7's raw-temperature posterior). A random-walk proposal is a small circle around the current point:

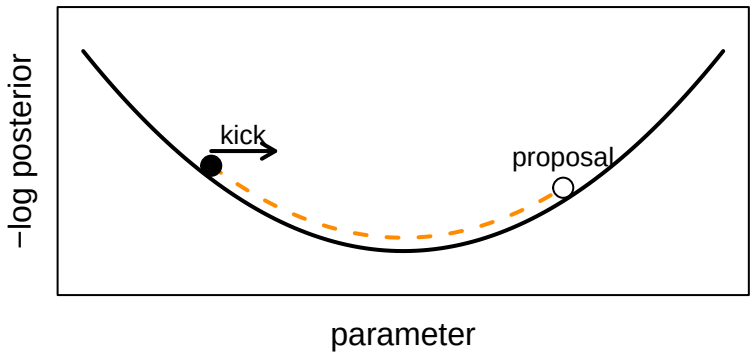
- Steps must be small enough to stay inside the *narrow* direction, so progress along the *long* direction is slow.
- A random walk covers distance only in proportion to $\sqrt{\text{steps}}$.
- With many parameters it gets worse: almost every random direction points "uphill."

BUGS/JAGS (Gibbs sampling). The first widely used tools (BUGS in the 1990s; JAGS later) update one parameter at a time from its conditional posterior. That works well when parameters are nearly independent, but on a tilted ellipse it zigzags along the axes, with the same slow progress.

R demo. Monday's myxo chain: 18,000 kept steps, but an effective sample size of only about 43 independent draws for shape. Then a correlated normal ($\rho = 0.95$): 200 random-walk steps, effective size about 6.

3. Hamiltonian Monte Carlo (~10 min) [BOARD → R]

Key idea. Turn the $-\log$ posterior into a landscape and slide a frictionless puck across it. The puck naturally follows the valley instead of stumbling across it.



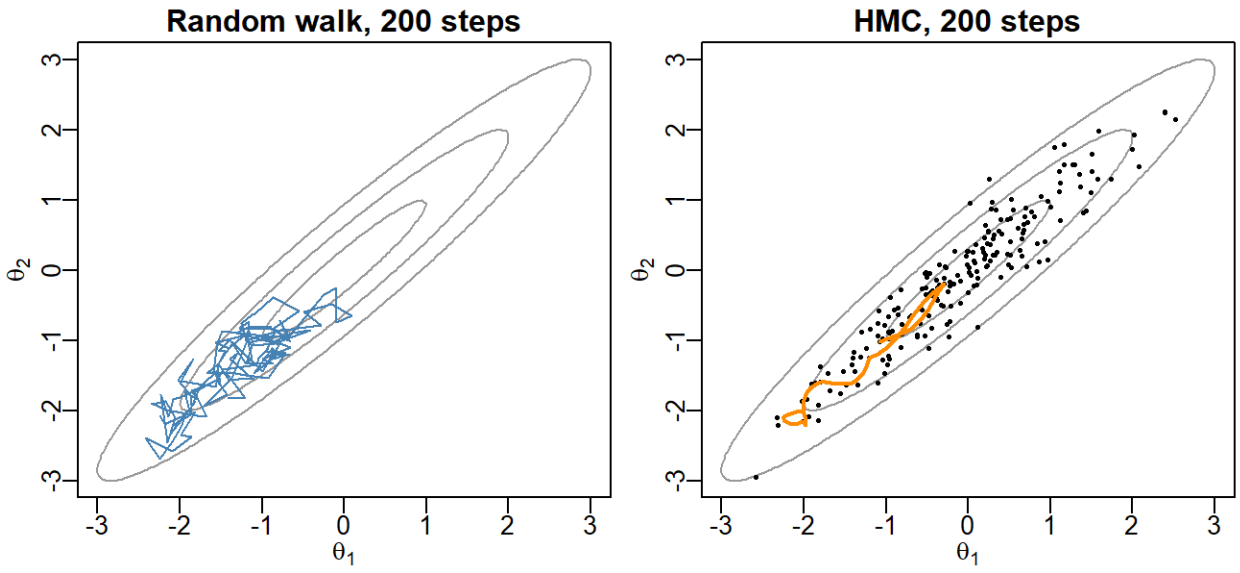
Board, step by step.

1. Give the puck a random **kick** (a momentum drawn from a normal).
2. Let it **slide** for L small time steps of size ϵ. Each step uses the **gradient** of the log posterior, so the puck speeds up going downhill and slows going uphill ("leapfrog" steps).
3. Where it stops is the **proposal**. Accept with the Metropolis rule on total energy; with small steps nearly everything is accepted.

- The gradient is what makes this work. Week 8 callback: Newton's method also used slopes, but to *find the bottom*; HMC uses them to *explore around* the bottom.
- Stan computes the gradient from your model code automatically (automatic differentiation): it applies the chain rule to each step of the calculation, so you never derive the gradient by hand, and it's exact, not a finite-difference approximation.
- **NUTS** (the "No-U-Turn Sampler," Stan's default) tunes ϵ during warmup and picks L on the fly, stopping when the puck starts to turn back.

Sticking point. Students hear "physics" and assume HMC is a different kind of inference. It isn't: it's still Metropolis (propose, then accept or reject), with proposals that are far away and still likely to be accepted.

R demo (skippable): the same 200 steps, random walk vs. HMC.



Effective size: random walk 6, HMC 183 out of 200. Each HMC step costs 11 gradient evaluations (one per leapfrog step, plus one at the start; each new gradient serves two half steps), and it's still far ahead. (In testing, effective size swung a lot with L; that sensitivity is why NUTS picks L automatically.)

4. Stan: the myxo model (~12 min) [R]

Walk through the model code (in the script): three blocks.

- `data`: what's fixed (N, the titers).
- `parameters`: what's sampled, with constraints (`<lower=0>`).
- `model`: the likelihood (and priors; none here, so flat on the allowed range, matching Monday).

Every variable is declared with a type; lines end in `;`; comments use `//`. Stan's `gamma()` takes shape and **rate**, so the code passes `1 / scale`.

Run it, then read the output.

- 4 chains × 1,000 draws (after 1,000 warmup draws each).
- $\hat{R}$ (R-hat) = 1.00: the chains agree (want below about 1.01).
- Effective size: about 452 for shape from 4,000 draws, vs. 43 from Monday's 18,000 Metropolis steps.
- Divergences: 0.
- 95% credible interval for shape: 25 to 76 (Monday's undersampled chain gave about 29 to 81).
- Plot the draws on Monday's surface: they fill the banana.

Cost. Compiling took about 8 s (once per model); sampling about 2 s.

5. When it fails; Stan vs. JAGS (~4 min) [BOARD]

- **Divergent transitions.** Where the surface curves sharply, the leapfrog steps overshoot and Stan flags a "divergence." Don't ignore them: the draws may be biased. Classic cause: the "funnel" in hierarchical models (returns in the multilevel weeks; the usual fix is reparameterization, as in Lab 7).
- **Discrete parameters.** Stan can't sample them (e.g. whether a site is truly occupied). You sum them out of the likelihood first, which is extra math. JAGS and NIMBLE handle these directly.
- **Multiple modes.** HMC still explores locally; well-separated valleys can trap it (run several chains from different starts).
- **Overhead.** Compiling, and a C++ toolchain to install. For a two-parameter model, anything works.
- Stan's payoff grows with the number of parameters and how correlated they are: multilevel models with hundreds of parameters.

6. Wrap, where to learn more (~2 min) [BOARD]

- **FIT**: grid → conjugacy → Metropolis → HMC. Same goal (the posterior), better engines.
- Learn more: Stan User's Guide and case studies (mc-stan.org); *Statistical Rethinking* Ch. 9; Betancourt (2017), "A Conceptual Introduction to Hamiltonian Monte Carlo" (arXiv:1701.02434); Monnahan et al. (2017); `brms` for regression models in Stan without writing Stan code.
- Hand out the take-home WOD.

Discussion (~5 min) [BOARD]

> For the myxo model, Monday's Metropolis sampler took 20,000 steps (under a second) and gave about 43 effective draws for shape. Stan took about 8 seconds to compile and 2 seconds to sample, and gave about 452 effective draws from 4,000. A labmate says Stan is overkill: "my sampler was faster." Who's right? For your own research, when would you reach for Stan, and when might JAGS or a hand-written sampler be the better choice?

Where to land. Compare effective draws per second, not steps per second; compiling happens once per model. For two parameters either tool is fine. Stan earns its keep as models grow (many parameters, strong correlations, hierarchical structure) and when diagnostics matter. JAGS or NIMBLE can be easier when the model has discrete latent states (occupancy, mark-recapture) that Stan would need marginalized.

PART 2: PODIUM CHEAT SHEET

Terse on purpose. Detail: Part 1. R = Wednesday section of CODE_Week9_MCMCAndStan.R (run Monday's section first; compile Stan before class).

No WOD. Debrief ~3 min, lecture ~38 min, discussion ~5 min. Behind? Skip the HMC run; shorten section 5.

0. Lab 7 debrief (~3 min) [R]

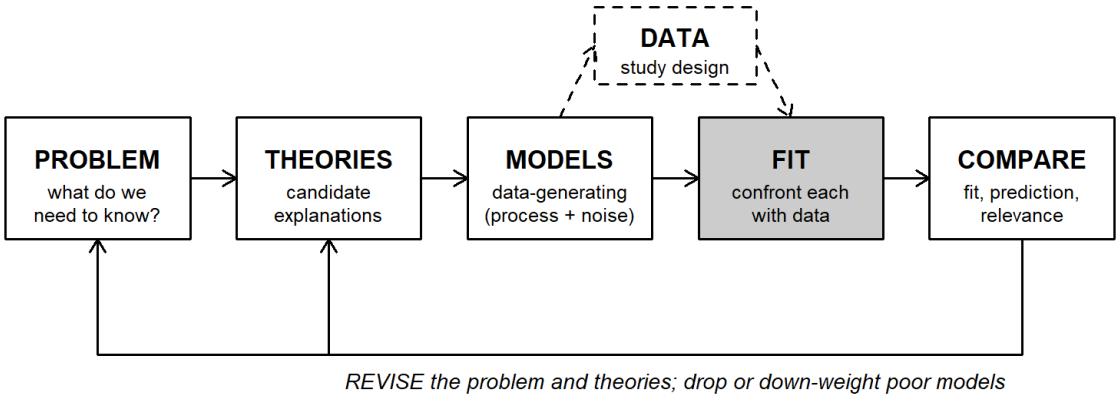
Start 0.05, 1,000 steps; exact mean 0.738:

- sd 0.005: 87% accepted, mean 0.38
- sd 0.1: 59%, mean 0.73
- sd 3: 4%, mean 0.70
- "First is best: nearly all accepted." Which Lab 7 chain is each?

Then show trace plots.

- Tiny steps: still in **burn-in** (Chains B, D)
- Huge steps: stuck (Chain C); close by luck
- Middle: **mixes** well
- Acceptance $\neq$ quality: trace plot, burn-in, several chains, tune width
- Segue: tilted ridge $\rightarrow$ even a tuned random walk creeps

1. Big picture (~3 min) [BOARD]

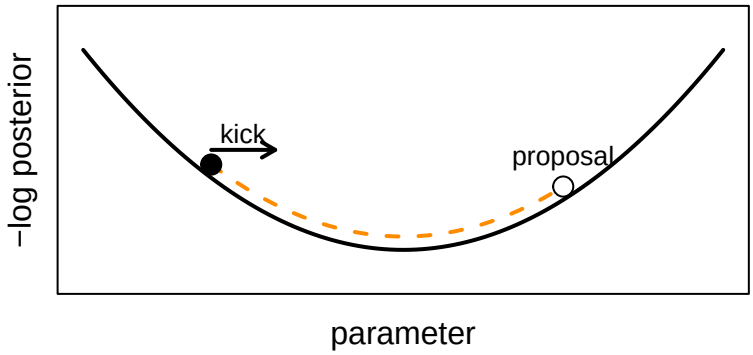


- **FIT**: Metropolis works but creeps (Lab 7)
- Today: use the shape of the surface; Stan builds it for you

2. Why random walks struggle (~7 min) [BOARD → R]

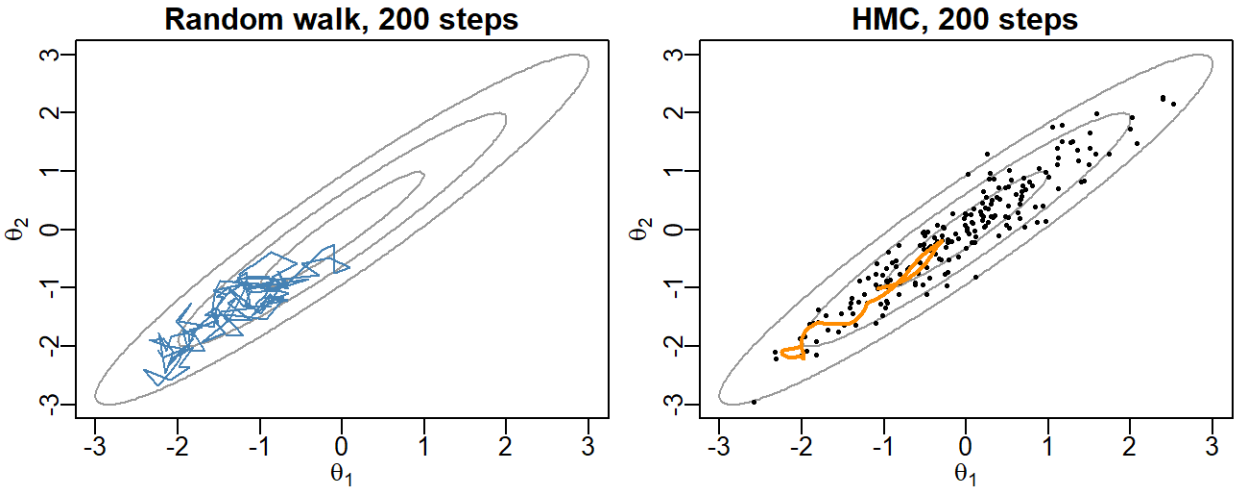
- Sketch tilted ellipse; small circle proposals
- Steps limited by narrow direction; distance $\propto \sqrt{\text{steps}}$
- Many parameters: most directions go uphill
- BUGS/JAGS: Gibbs, one parameter at a time → zigzag
- **R:** myxo 18,000 steps → ESS 43; $\rho = 0.95$: 200 steps → ESS 6

3. HMC (~10 min) [BOARD → R]



1. Random **kick** (momentum)
2. **Slide** L leapfrog steps using the **gradient**
3. **Accept/reject** on energy (almost always accept)

- Newton finds the bottom; HMC explores around it
- Autodiff: chain rule applied automatically; exact gradients, no hand derivation
- NUTS: tunes ϵ, picks L
- Warmup learns each parameter's scale (the mass matrix)
- **Still Metropolis**, with far-reaching proposals



- **R (skippable):** ESS 6 vs. 183 of 200

4. Stan: myxo (~12 min) [R]

- Blocks: `data`, `parameters` (`<lower=0>`), `model`
- Types; `;`; `//`; gamma(shape, **rate**) → `1 / scale`
- 4 chains × 1,000; $\hat{R} = 1.00$; divergences 0
- ESS shape 452 (vs. 43)
- Shape 95%: 25-76 (Monday ~29-81)
- Plot draws on surface: fills the banana
- Compile ~8 s once; sample ~2 s

5. Failures; vs. JAGS (~4 min) [BOARD]

- **Divergences**: sharp curvature; funnels; reparameterize
- **Discrete parameters**: marginalize (JAGS/NIMBLE direct)
- Multiple modes: several chains
- Overhead: compile, toolchain
- Payoff grows with # parameters and correlation

6. Wrap (~2 min) [BOARD]

- Grid → conjugacy → Metropolis → HMC
- mc-stan.org; Rethinking Ch. 9; Betancourt 2017; Monnahan et al. 2017; brms
- **Hand out take-home WOD**

Discussion (~5 min)

- Metropolis: <1 s, ESS 43; Stan: 8 s compile + 2 s, ESS 452
- "Overkill?" When Stan, when JAGS?
- ESS per second; compile once; payoff with size; discrete latent states → JAGS/NIMBLE